

Task di implementazione progetto rubricaJavaAngular

Questo documento suddivide l'implementazione dell'applicazione in task piccoli, verificabili e committabili.

Ogni task deve rispettare il seguente ciclo:

1. Implementazione mirata.
2. Verifica assenza errori IDE.
3. Esecuzione build/test pertinenti.
4. Generazione report test/coverage quando il task riguarda test o qualita'.
5. Correzione eventuali errori.
6. Aggiornamento versione frontend e backend, se il task viene pubblicato su GitHub.
7. Commit finale con messaggio chiaro.

Prima di ogni commit pubblicato su GitHub devono essere aggiornate entrambe le versioni:

- **frontend:** frontend/package.json;
- **backend:** backend/build.gradle o backend/build.gradle.kts, usando la proprieta' Gradle standard version.

Fase 1 - Setup repository

Task 1.1 - Preparazione struttura repository

Obiettivo:

- rinominare o creare il progetto locale come rubricaJavaAngular;
- creare struttura base:

```
frontend/  
backend/  
mockup/  
README.md  
REQUISITI_RUBRICA_FULLSTACK.md  
PROMPT_STORICO.md  
TASK_IMPLEMENTAZIONE.md
```

Verifiche:

- controllare che la struttura sia coerente;
- controllare assenza file temporanei;
- controllare `.gitignore`.

Correzione:

- sistemare nomi cartelle o file non coerenti.

Commit:

```
chore: initialize project structure
```

Task 1.2 - Configurazione GitHub

Obiettivo:

- collegare repository remoto `https://github.com/Michelepal/rubricaJavaAngular`;
- verificare branch principale;
- documentare remote.

Verifiche:

- `git remote -v`;
- nessun file sensibile tracciato.

Correzione:

- aggiornare `.gitignore`;
- rimuovere eventuali file generati non necessari.

Commit:

```
chore: configure repository metadata
```

Fase 2 - Backend Spring Boot

Task 2.1 - Creazione progetto Gradle Spring Boot

Obiettivo:

- creare progetto backend Gradle;
- configurare `settings.gradle` con `rootProject.name = 'rubricaJavaAngular'`;
- configurare `group` e `version`;
- aggiungere Gradle Wrapper.

Verifiche:

- import corretto nell'IDE;
- assenza errori Gradle;
- esecuzione:

```
./gradlew clean build
```

Correzione:

- correggere dipendenze, plugin o struttura Gradle.

Commit:

```
chore(backend): create spring boot gradle project
```

Task 2.2 - Configurazione profili dev/prod

Obiettivo:

- creare `application.yml`;
- creare `application-dev.yml`;
- creare `application-prod.yml`;
- configurare log dev dettagliati e prod essenziali;
- configurare H2 dev;
- disabilitare H2 console in prod.

Verifiche:

- avvio backend con profilo `dev`;
- avvio backend con profilo `prod`;
- nessun errore IDE;
- nessun segreto committato.

Correzione:

- correggere properties errate;
- sistemare logging o profili.

Commit:

```
chore(backend): configure spring profiles
```

Task 2.3 - Modello dati JPA

Obiettivo:

- creare entity:
 - User;
 - Contact;
 - ContactPhone;
 - ContactEmail;
 - ContactAddress;
 - Tag;
- configurare relazioni e vincoli.

Verifiche:

- assenza errori IDE;
- test repository minimi;
- avvio con H2;
- verifica schema generato.

Correzione:

- sistemare annotazioni JPA, cascade, fetch e vincoli.

Commit:

```
feat(backend): add address book domain model
```

Task 2.4 - Repository JPA

Obiettivo:

- creare repository per utenti, contatti e tag;
- aggiungere query filtrate per utente autenticato.

Verifiche:

- test repository con H2;
- verifica vincoli unique;
- verifica isolamento dati tra utenti.

Correzione:

- correggere query o metodi repository.

Commit:

```
feat(backend): add jpa repositories
```

Task 2.5 - DTO, mapper e validazioni backend

Obiettivo:

- creare DTO request/response;
- aggiungere Bean Validation;
- creare mapper dedicati;
- evitare esposizione entity nelle API.

Verifiche:

- test validator DTO;
- test mapper;
- assenza errori IDE;
- coverage generata.

Correzione:

- correggere pattern, size, messaggi e mapping.

Commit:

```
feat(backend): add dto validation and mapping
```

Task 2.6 - Gestione centralizzata errori REST

Obiettivo:

- creare modello errore REST;
- creare `@ControllerAdvice`;
- gestire 400, 401, 403, 404, 409, 500;
- messaggi comprensibili e sicuri.

Verifiche:

- test controller advice;
- test errori di validazione;
- nessuno stack trace nella response;
- log tecnici solo lato server.

Correzione:

- correggere formato errori e status HTTP.

Commit:

```
feat(backend): add rest error handling
```

Task 2.7 - Spring Security e JWT

Obiettivo:

- configurare Spring Security;
- implementare login;
- implementare JWT service;
- proteggere `/api/**`;
- configurare CORS dev.

Verifiche:

- test login riuscito;
- test login fallito;
- test token valido/non valido;
- test endpoint protetto senza token;
- assenza errori IDE.

Correzione:

- correggere filtri, security chain, CORS o password encoder.

Commit:

```
feat(backend): add spring security jwt authentication
```

Task 2.8 - API contatti

Obiettivo:

- implementare CRUD contatti;
- filtrare sempre per utente autenticato;
- gestire telefoni, email e indirizzi.

Verifiche:

- test service;
- test controller;
- test isolamento dati tra utenti;
- test errori REST;
- coverage JaCoCo.

Correzione:

- correggere ownership, mapping e validazioni.

Commit:

```
feat(backend): add contact rest api
```

Task 2.9 - API tag

Obiettivo:

- implementare CRUD tag;
- modificare tag;
- cancellare tag;
- garantire unique (user_id, name);
- impedire accesso a tag di altri utenti.

Verifiche:

- test modifica tag;
- test cancellazione tag;
- test duplicato tag;
- test ownership;
- test errori REST.

Correzione:

- correggere service, repository o vincoli.

Commit:

```
feat(backend): add tag management api
```

Task 2.10 - Fallback SPA e risorse statiche

Obiettivo:

- configurare Spring Boot per servire Angular da resources/static;
- aggiungere fallback SPA verso index.html;
- escludere /api/** dal fallback.

Verifiche:

- test apertura /;
- test apertura /login;
- test apertura /home;
- test /api/** non intercettato;

- build backend.

Correzione:

- correggere controller fallback o security config.

Commit:

```
feat(backend): serve angular spa
```

Fase 3 - Frontend Angular

Task 3.1 - Creazione progetto Angular

Obiettivo:

- creare progetto Angular in `frontend/`;
- configurare routing;
- configurare environment dev/prod;
- configurare script test e build.

Verifiche:

- import corretto IDE;
- `npm install`;
- `npm test`;
- `npm run build`;
- nessun errore TypeScript.

Correzione:

- correggere configurazione Angular, tsconfig o dipendenze.

Commit:

```
chore(frontend): create angular project
```

Task 3.2 - Layout base responsive

Obiettivo:

- creare layout principale;
- creare login, home, error page;
- definire breakpoint responsive;
- impostare font e scala tipografica.

Verifiche:

- test componenti;
- verifica manuale mobile/tablet/desktop;
- nessun overflow orizzontale;
- nessun errore IDE.

Correzione:

- sistemare CSS, layout e breakpoint.

Commit:

```
feat(frontend): add responsive base layout
```

Task 3.3 - Tema chiaro/scuro

Obiettivo:

- implementare switch light/dark;
- salvare preferenza;
- rispettare preferenza di sistema dove opportuno.

Verifiche:

- test servizio tema;
- test persistenza preferenza;
- verifica contrasto light/dark;
- verifica componenti principali in entrambi i temi.

Correzione:

- correggere variabili CSS, local storage o stati UI.

Commit:

```
feat(frontend): add light dark theme switch
```

Task 3.4 - Accessibilita' UI

Obiettivo:

- label form corrette;
- focus visibile;
- navigazione tastiera;
- aria-label per pulsanti iconici;

- modali accessibili.

Verifiche:

- test componenti;
- verifica navigazione tastiera;
- verifica screen reader dove possibile;
- controllo contrasto.

Correzione:

- correggere markup, ARIA, focus e contrasto.

Commit:

```
feat(frontend): improve accessibility
```

Task 3.5 - Auth service, interceptor e guard

Obiettivo:

- implementare `AuthService`;
- implementare interceptor JWT;
- implementare guard rotte protette;
- gestire logout.

Verifiche:

- test login riuscito;
- test login fallito;
- test token in header;
- test guard;
- test errori 401 e 403.

Correzione:

- correggere gestione token, interceptor o routing.

Commit:

```
feat(frontend): add authentication flow
```

Task 3.6 - Gestione errori frontend

Obiettivo:

- creare servizio gestione errori;

- mappare errori REST in messaggi utente;
- gestire errori form e pagina errore.

Verifiche:

- test 400, 401, 403, 404, 409, 500;
- test errore rete;
- verifica messaggi comprensibili;
- verifica accessibilità messaggi.

Correzione:

- correggere mapping errori e UI.

Commit:

```
feat(frontend): add rest error handling
```

Task 3.7 - Validator e sanitizer frontend

Obiettivo:

- implementare validator custom;
- implementare normalizzazione dati;
- sanificare input testuali.

Verifiche:

- test validator nomi;
- test validator telefono;
- test validator tag;
- test sanitizer;
- coverage frontend.

Correzione:

- correggere pattern e messaggi.

Commit:

```
feat(frontend): add validators and sanitizers
```

Task 3.8 - Lista contatti

Obiettivo:

- mostrare contatti;

- aggiungere ricerca e filtri;
- gestire stati loading/error/empty.

Verifiche:

- test service contatti;
- test componente lista;
- verifica responsive;
- verifica errori API.

Correzione:

- correggere rendering, servizi o stati UI.

Commit:

```
feat(frontend): add contacts list
```

Task 3.9 - Form contatto

Obiettivo:

- creare form creazione/modifica;
- gestire telefoni, email, indirizzi;
- validare e normalizzare dati.

Verifiche:

- test form valido/non valido;
- test submit;
- test normalizzazione;
- test errori backend.

Correzione:

- correggere form model, validator o mapping.

Commit:

```
feat(frontend): add contact form
```

Task 3.10 - Modali di conferma

Obiettivo:

- modale conferma modifica;
- modale conferma cancellazione;

- focus trap e accessibilita';
- nessuna chiamata API se annullata.

Verifiche:

- test apertura modale;
- test conferma;
- test annullamento;
- test focus;
- verifica responsive.

Correzione:

- correggere gestione stato, focus o azioni.

Commit:

```
feat(frontend): add confirmation modals
```

Task 3.11 - Gestione tag frontend

Obiettivo:

- visualizzare tag;
- modificare tag;
- cancellare tag;
- modale conferma per modifica/cancellazione;
- validare nome tag.

Verifiche:

- test modifica tag;
- test cancellazione tag;
- test modale;
- test errore duplicato 409;
- test accessibilita'.

Correzione:

- correggere servizi, form tag o modali.

Commit:

```
feat(frontend): add tag management
```

Fase 4 - Integrazione full stack

Task 4.1 - Collegamento frontend-backend in dev

Obiettivo:

- configurare API base URL dev;
- configurare CORS o proxy;
- verificare login reale;
- verificare CRUD contatti e tag.

Verifiche:

- backend dev avviato;
- frontend Angular avviato;
- login;
- lista contatti;
- creazione/modifica/cancellazione;
- gestione errori.

Correzione:

- correggere CORS, URL API o mapping DTO.

Commit:

```
feat: integrate frontend and backend in development
```

Task 4.2 - Build Angular dentro backend

Obiettivo:

- generare build Angular production;
- copiare artifact in backend/src/main/resources/static;
- verificare fallback SPA.

Verifiche:

- npm run build;
- ./gradlew bootJar;
- avvio jar con profilo prod;
- apertura /;
- apertura /login;
- apertura /home;
- chiamate /api/**.

Correzione:

- correggere script build, path statici o fallback.

Commit:

```
feat: integrate angular build into spring boot
```

Task 4.3 - Test end-to-end manuale

Obiettivo:

- verificare flusso utente completo.

Verifiche:

- login valido;
- login errato;
- creazione contatto;
- modifica contatto;
- cancellazione contatto con modale;
- modifica tag con modale;
- cancellazione tag con modale;
- switch tema;
- responsive;
- accessibilita' base;
- browser moderni principali dove disponibili.

Correzione:

- correggere bug emersi.

Commit:

```
test: validate full user flow
```

Fase 5 - Qualita', coverage e rilascio

Task 5.1 - Coverage backend

Obiettivo:

- configurare JaCoCo;
- generare report coverage;
- generare report sullo stato test backend;

- aumentare copertura parti critiche.

Verifiche:

- task Gradle test e report coverage;
- report test consultabile in `backend/build/reports/tests/`;
- report coverage consultabile in `backend/build/reports/jacoco/`;
- copertura service/security/controller/validator;
- assenza errori IDE.

Correzione:

- aggiungere test mancanti;
- correggere codice non testabile.

Commit:

```
test(backend): add coverage reporting
```

Task 5.2 - Coverage frontend

Obiettivo:

- generare report coverage Angular;
- generare report sullo stato test frontend;
- aumentare copertura parti critiche.

Verifiche:

- test frontend;
- report coverage consultabile in `frontend/coverage/`;
- coverage validator, service, guard, interceptor, modali;
- nessun errore TypeScript.

Correzione:

- aggiungere test mancanti;
- correggere codice non testabile.

Commit:

```
test(frontend): add coverage reporting
```

Task 5.3 - Revisione documentazione

Obiettivo:

- aggiornare README;
- aggiornare requisiti;
- aggiornare storico prompt;
- documentare avvio dev/prod;
- documentare versionamento.

Verifiche:

- link e comandi corretti;
- nessuna informazione obsoleta;
- nessun file sensibile.

Correzione:

- correggere documentazione.

Commit:

```
docs: update project documentation
```

Task 5.4 - Preparazione pubblicazione GitHub

Obiettivo:

- verificare stato repository;
- eseguire build e test completi;
- aggiornare versioni frontend e backend;
- preparare commit finale.

Verifiche:

```
git status
```

Backend:

```
./gradlew clean build
```

Frontend:

```
npm test  
npm run build
```

Correzione:

- correggere ogni errore IDE, build o test;

- ripetere verifiche.

Commit:

```
chore: prepare github publication
```

Push:

```
git push origin <branch>
```

Regole finali per ogni task

- Non procedere al task successivo se il task corrente lascia errori IDE evidenti.
- Non committare codice che non compila.
- Non committare test falliti.
- Non pubblicare segreti o artifact locali non previsti.
- Ogni commit GitHub deve aggiornare versione frontend e backend.
- Ogni modifica deve rispettare SOLID, DRY, naming chiaro e struttura standard Angular/Spring Boot/Gradle.
- Ogni errore REST deve produrre messaggi comprensibili all'utente.
- Ogni modifica o cancellazione dati deve passare da modale di conferma.